m1

Noise-Based Terrain | Procedural Landscape Generation in Three.js

Lesson Objective

This lesson explains how realistic terrain is generated in a Three.js-based Fantasy Map Engine using noise functions instead of pure randomness. The goal is to understand why Perlin or Simplex noise is required for natural-looking landscapes.

https://www.udemy.com/course/web-based-3d-terrain-industrial-printing-engine-mastery/

Core Concept: Randomness vs Noise

There are two approaches to terrain generation:

1. Pure Randomness

Each vertex is assigned an independent value.

- No relationship between neighbors
- No structure in the surface
- Chaotic results

Result:

- Sharp spikes
- Broken geometry
- Unnatural terrain

Problem: No spatial continuity

2. Noise-Based Generation

- Values change smoothly over space
- Terrain becomes continuous

Result:

- Hills
- Mountains
- Valleys
- Natural landscapes

Key idea: spatial continuity

Why Randomness Fails

Using Math.random() for terrain:

- Produces disconnected vertices
- Creates jagged surfaces
- Breaks realism
- Removes structure

Conclusion: randomness is unusable for terrain systems

Why Noise Works

Noise functions (Perlin / Simplex) provide:

- Smooth transitions
- Gradual elevation changes
- Natural variation patterns

Result:

- Organic terrain shapes
- Realistic geography
- Continuous landscapes

Conclusion: noise introduces structured randomness

We start with PlaneGeometry.

Each vertex contains:

- x position
- y height
- z position

Stored in BufferGeometry position array.

Step 2: Noise Input Mapping

For each vertex:

We use:

- x coordinate
- z coordinate

These are passed into the noise function:

noise(x, z)

Output: height value

Step 3: Height Assignment

Noise output is assigned to:

vertex.y

Transformation:

Flat grid → 3D terrain

Step 4: Smoothness Principle

Noise ensures:

- Nearby coordinates produce similar values
- Distant coordinates differ more

- No sudden jumps
- Natural surface transitions

Step 5: Random vs Noise Comparison

Random Terrain:

- Disconnected points
- Sharp spikes
- No structure

Noise Terrain:

- Smooth gradients
- Organic shapes
- Realistic landscapes

Key difference: correlation between vertices

Step 6: Frequency Control

Input coordinates are scaled.

High frequency:

- More variation
- Rough terrain

Low frequency:

- Smooth terrain
- Rolling hills

Frequency controls terrain detail density

Step 7: Amplitude Control

Amplitude defines height intensity.

Controls:

Without it, terrain lacks vertical control.

Step 8: BufferGeometry Update

After modifying vertices:

- Update position array
- Mark geometry as needing update

Result:

- GPU receives updated data
- Scene refreshes instantly

Step 9: Real-Time Terrain Generation

System works dynamically:

- Noise calculated per vertex
- Geometry updates instantly
- WebGL renders changes immediately

Result: live procedural terrain

Step 10: Procedural Foundation

Noise-based terrain is used in:

- World generation systems
- Biomes
- Erosion systems
- Procedural maps

Terrain is generated mathematically, not manually.

System Flow Summary

1. Iterate vertices

4. Compute height
5. Assign to y
6. Update geometry
7. Mark for GPU update
8. Render result

Key Concept Shift

Random Generation:

- Chaos
- No structure
- No continuity

Noise-Based Generation:

- Structured variation
- Smooth transitions
- Natural terrain

Noise introduces order inside randomness.

System Summary

After this lesson:

- Random values are unsuitable for terrain
- Noise functions create natural landscapes
- Perlin/Simplex ensures continuity
- x and z define input space
- y stores elevation
- Frequency controls detail
- Amplitude controls height
- BufferGeometry enables real-time updates
- Terrain is fully mathematical

Student Tasks

Task 1: Random vs Noise

Compare Math.random() terrain with noise terrain.

Feed x and z into noise function.

Task 3: Frequency Test

Increase scale and observe detail changes.

Task 4: Amplitude Test

Adjust height multiplier and compare results.

Task 5: Real-Time Update

Modify parameters and observe instant updates.

Final Outcome

By the end:

- Noise replaces randomness in terrain generation
- Natural landscapes are generated mathematically
- Real-time GPU rendering is enabled
- Frequency and amplitude shape world structure
- Full procedural terrain becomes possible in the browser

m2